Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

CS101

Building from scratch

Today
Welcome to your next lesson: Building from scratch.

I'm preparing your personalized lesson by looking at what you've learned, what's ahead, and the best way to help you grow.

Step by step, you’re moving forward.

Imagine you’re building a simple workout tracker 💪

You have a list of workouts for the week, and for each workout you want to clean up the data and compute total minutes.

There are two ways to structure this:

Pattern A:

```
for workout in workouts:
    clean_and_summarize(workout)
```

Pattern B:

```
weekly_summary = summarize_week(workouts)
```

Both will use a loop and a helper function — but in A the loop is outside, in B the loop is inside the helper.

Quick check:

Which pattern feels clearer to you for a weekly workout summary, A or B? (No need to justify yet, just pick one.)

Pattern B

Nice, Pattern B is a very natural choice for “summarize the whole week” 👍

Let’s zoom in on what Pattern B usually means in code.

A typical version looks like this:

Python

```
def summarize_week(workouts):
    total_minutes = 0
    for workout in workouts:
        # do something with each workout
        total_minutes += workout["minutes"]
    return total_minutes
```

Key idea (one concept):

The loop lives inside summarize_week, and from the outside you just call it once with the whole list.

So the caller doesn't see the loop; it just says "hey, summarize this list for me."

Does that idea make sense: one call with a list → helper owns the loop inside?

If yes, I'll give you a small prediction question next about how the data flows through this.

Yes, that makes sense

What do you think? Take your time with the question below 👇

Given this function:

```
def summarize_week(workouts):
    total_minutes = 0
    for workout in workouts:
        total_minutes += workout["minutes"]
    return total_minutes
```

and this data:

```
workouts = [
    {"minutes": 30},
    {"minutes": 45},
    {"minutes": 15},
]

result = summarize_week(workouts)
```

Where does the loop actually run?

A.

Inside summarize_week, hidden from the caller

B.

Outside, in the code that calls summarize_week

C.

There is no loop here, only addition

D.

It depends on what workouts contains at runtime

Exactly right 🎯

The loop runs inside summarize_week. From the outside, you just do:

Python

```
result = summarize_week(workouts)
```

That line runs the whole function once, and inside that function the for workout in

workouts: loop walks through the list and adds up the minutes. The calling code never sees the loop details — it just gets the final result.

Next, I want to show you the other pattern side-by-side (loop outside, helper inside).

Ready to look at that version?

Yes, I'm ready

Great 👍

Now let’s look at the other structure: loop outside, helper inside.

Same workout idea, but written like this:

Python

```
def workout_minutes(workout):
    return workout["minutes"]

total = 0
for workout in workouts:
    total += workout_minutes(workout)
```

Step by step:

The loop is in the main code (for workout in workouts:).
Inside the loop, we call the helper once per item: workout_minutes(workout).
The helper only deals with one workout at a time, not the whole list.

So here, the caller owns the loop, and the helper just handles a single piece.

Does this contrast feel clear so far?

Pattern B (earlier): helper owns the loop, takes a list.
This pattern: main code owns the loop, helper takes one item.
Yes, it's clear

Nice 👌 That’s the core distinction.

Let’s now focus on how data moves when we put a helper inside a loop.

Look at this version:

Python

```
def workout_minutes(workout):
    return workout["minutes"]

workouts = [
    {"minutes": 30},
    {"minutes": 45},
    {"minutes": 15},
]

total = 0
for workout in workouts:
    minutes = workout_minutes(workout)
    total += minutes
```

If we trace it step by step, each loop does:

Take one workout from the list
Call workout_minutes(workout) → get a number

Add that number into total

So each iteration is like: item in → function → result out → added to total.

Quick check before we build a little trace table together:

Does that 3-step flow per loop iteration make sense to you?

Yes, it makes sense

What do you think happens on that second iteration? Look at the options below 👇

On the second iteration of this loop, what is passed into the function workout_minutes?

```
def workout_minutes(workout):
    return workout["minutes"]

workouts = [
    {"minutes": 30},
    {"minutes": 45},
    {"minutes": 15},
]

for workout in workouts:
    minutes = workout_minutes(workout)
```

A.

The whole list workouts

B.

Only the current workout dictionary for that iteration

C.

Just the number of minutes like 30 or 45

D.

Nothing is passed; the function reads a global variable

Nice job, Ron 🎯 You nailed it.

On each iteration, workout refers to just one dictionary from the list, so the function call:

Python

minutes = workout_minutes(workout)
passes only the current workout dictionary (like {"minutes": 45} on the second iteration). The function never sees the whole list, only one item at a time, which is exactly how helpers-inside-loops are meant to work.

Next step: I want to show you the same data flow but with the loop inside the helper and compare the two.

Ready to see that side□by□side?

Yes, I'm ready

Cool, let’s line them up side by side 👀

Same goal: total minutes for the week.

Pattern 1 – loop OUTSIDE, helper per item:

Python

```
def workout_minutes(workout):
    return workout["minutes"]

total = 0
for workout in workouts:
    total += workout_minutes(workout)
```

Pattern 2 – loop INSIDE the helper:

Python

```
def summarize_week(workouts):
    total = 0
    for workout in workouts:
        total += workout["minutes"]
    return total

total = summarize_week(workouts)
```

Visually:

Pattern 1: for each workout → call helper(workout)
Pattern 2: call helper(workouts) → inside it: for each workout ...

Both give the same result, but the loop “lives” in a different place.

I want to check your feel for when you might prefer each.

Pick the option that best matches Pattern 2 (loop inside helper) in the question below 👇

id prefer the loop in the main code if i needed to use each ite for something additional like
for workout in workouts"
total += workout["minutes"]
workout_count += 1
if prefer the loop inside the function if i dont need to process each item and just want the tota

That’s an excellent instinct, and your reasoning is spot on 👌

You’re basically saying:

Loop in main code when you want to do multiple things with each item, like:

```
for workout in workouts:
    total_minutes += workout["minutes"]
    workout_count += 1
    # maybe more: print, filter, etc.
```

Loop inside function when you just want a single result (like total) and don’t care about each item separately:

```
total_minutes = summarize_week(workouts)
```

That’s exactly the kind of design decision this lesson is about.

Let’s make this concrete in code next.

Would you like to:

A) Practice the loop in main code version first, or
B) Practice the loop inside function version first?
a

Great choice 👍 Let’s practice loop in main code.

Here’s the pattern we’ll use, step by step:

Python

```
def workout_minutes(workout):
    return workout["minutes"]

workouts = [
    {"minutes": 20},
    {"minutes": 35},
    {"minutes": 40},
]

total_minutes = 0
for workout in workouts:
    minutes = workout_minutes(workout)  # 1) call helper with ONE item
    total_minutes += minutes            # 2) update accumulator
```

Visually for each loop:

one workout dict → workout_minutes(...) → number of minutes → add into total

Your turn (in the code editor, not in chat):

Open the editor (</> button).
Recreate this pattern, but also keep a workout_count and increment it inside the loop.
After the loop, print both total_minutes and workout_count with clear labels (use f-strings).

When you're happy, hit Submit code and I'll review it.

Python

```
def workout_minutes(workout):
    return workout["minutes"]

workouts = [
    {"minutes": 20},
    {"minutes": 35},
    {"minutes": 40},
]

workout_count = 0
total_minutes = 0
for workout in workouts:
    minutes = workout_minutes(workout)  # 1) call helper with ONE item
    total_minutes += minutes            # 2) update accumulator
    workout_count += 1
print(f"Total minutes was: {total_minutes}, Total numer of workouts was: {workout_count}")
```

Nice work, Ron

You've done exactly what we wanted for loop in main code + helper:

workout_minutes is a clean helper that handles one workout.
The for workout in workouts: loop:
calls the helper each time,
keeps total_minutes as an accumulator,
keeps workout_count as a second accumulator.
Your final print clearly shows both values: 95 minutes and 3 workouts.

This is a perfect example of "loop outside, helper per item."

Next, let's flip it: same workouts list, but now write a summarize_week(workouts) function that:

Owns the for loop inside the function,
Computes and returns the total minutes,
And in main code you just do:
Python

```
total_minutes = summarize_week(workouts)
print(total_minutes)
```

Go ahead and build that version in the code editor, then Submit code and I'll check it.

Python

```
def summaraize_week(workouts):
    total_minutes = 0
    for workout in workouts:
        total_minutes += workout["minutes"]
    return total_minutes

workouts = [
    {"minutes": 20},
    {"minutes": 35},
    {"minutes": 40},
]

total_minutes = summaraize_week(workouts)
print(total_minutes)
```

Beautifully done 👌

You just implemented the "loop inside the helper" pattern perfectly:

Python

```
def summaraize_week(workouts):
    total_minutes = 0
    for workout in workouts:
        total_minutes += workout["minutes"]
    return total_minutes
```

The function takes the whole list.
It owns the for loop.
It returns the final total_minutes.
The calling code is super simple:
Python

```
total_minutes = summaraize_week(workouts)
print(total_minutes)
```

So now you've correctly used both patterns:

loop in main code + helper per item
loop inside helper + single call from main
Let's check your understanding with a quick question about when to pick each style.

Look at the options below and pick the best answer 👇

your question and list of options did not appear

What do you think fits best here? Pick an option below 👇

You have a list of workouts and you need to (1) total the minutes and (2) also log each workout individually with print inside the loop.

Which structure is usually the better design?

A.

Put the loop inside a helper like summarize_week(workouts) and only return the total

B.

Keep the loop in the main code, and call a helper for each workout if needed

C.

Avoid helpers and write everything directly in one big loop

D.

It doesn't matter; both designs are identical in how flexible they are

Nice catch earlier about the missing question, and nice answer once it showed up 🎯

You were right:

When you need to do several things with each item (like total minutes and print each workout), it's better to:
Keep the loop in the main code, and
Optionally call helpers inside that loop.
That keeps things flexible: you can add more per□item logic later without changing a big "black box" function.

Putting the loop inside summarize_week is great when you just want a single result (like a total) and don't care about each workout individually.

You've now:

Written code with loop outside + helper per item
Written code with loop inside helper + single call
Explained when to choose which ☑

There's one more idea I want to lock in: tracing how data moves through a loop + helper using a small table (iteration → input → output). After that, you'll basically have the whole lesson.

Want to do a quick trace-table exercise next?

Yes, let's try it

Nice 🔍 Let's keep this very concrete.

We'll trace this code:

Python

```
def workout_minutes(workout):
    return workout["minutes"]

workouts = [
    {"minutes": 10},
    {"minutes": 20},
    {"minutes": 30},
]

total = 0
for workout in workouts:
    minutes = workout_minutes(workout)
    total += minutes
```

We can track the data in a little table like this:

iteration	workout	minutes	total after line total += minutes
1	{"minutes": 10}	10	10
2	{"minutes": 20}	20	30
3	{"minutes": 30}	30	60

Key idea: each row is one loop run: item in → helper result → updated total.

Your turn, just thinking in chat:

On iteration 2, what do you expect the value of total to be right after total += minutes runs?

30

Exactly ☑

On iteration 2:

minutes is 20
previous total was 10
total += minutes → total becomes 30

You clearly understand how the data flows through the loop + helper combo.

At this point, you've shown you can:

Use helpers inside loops
Put loops inside helpers
Trace data flow across iterations
Decide where the loop should live based on what you need

You've basically mastered this lesson's goals 🎯

Are you ready to wrap up this lesson and move on to the next one, or would you like one more quick challenge with a fresh example (like steps tracker or inventory) before we finish?

I'm ready to move on

Awesome work on this lesson, Ron 🎓

Quick recap of what you can now do:

Loop in main code + helper per item
You wrote patterns like:

```
for workout in workouts:
    minutes = workout_minutes(workout)
    total_minutes += minutes
```

Great when you need to do multiple things per item.

Loop inside helper + single call
You wrote:

```
total_minutes = summaraize_week(workouts)
```

Perfect when you just want a single result and don't care about each item separately.

Trace data flow across iterations
You followed how workout → minutes → total changed on each loop run, which is a big part of real debugging and reasoning about code.

You're thinking like a designer now, not just a coder — choosing where the loop "lives" based on what the program needs. That's a pro move 🙌

When you're ready, you can jump into the next lesson from the Learn tab, and I'll be right here to keep building on this.

```
def summaraize_week(workouts):
```

```
    total_minutes = 0
    for workout in workouts:
        total_minutes += workout["minutes"]
    return total_minutes

workouts = [
    {"minutes": 20},
    {"minutes": 35},
    {"minutes": 40},
]

total_minutes = summaraize_week(workouts)
print(total_minutes)
```